Planejamento Lógico TaskTracker — Bruno Mateus Borges

Bootcamp II | Desafio Cumulativo: Arquitetura de Dados, Algoritmo e Mapeamento Git/GitHub

1. Apresentação e Entendimento do Problema

Campo de Identificação	Dados do Estudante
Nome Completo	Bruno Mateus Borges
Curso	Ciência da Computação
Instituição e Polo/Turno	Centro Universitário de Brasília (CEUB) — Campus Asa Norte (Noturno)
Disciplina	Bootcamp II (EAD — Disciplina Virtual)
E-mail de Contato	brunomateusb60@gmail.com

O Problema a Ser Resolvido

No cotidiano acadêmico e profissional, estudantes e equipes de desenvolvimento lidam simultaneamente com múltiplas demandas, entregas de projetos e prazos rigorosos. Quando o controle dessas atividades é feito de maneira improvisada — por meio de anotações soltas em papel, mensagens dispersas ou apenas na memória — surgem falhas críticas: esquecimento de datas importantes, ausência de critério claro sobre o que deve ser executado primeiro e retrabalho constante. A falta de padronização nas informações registradas impede a visualização rápida do que está pendente, gerando sobrecarga cognitiva e queda direta na produtividade.

A Solução Proposta (TaskTracker)

O **TaskTracker** é uma aplicação de gerenciamento de tarefas via Interface de Linha de Comando (CLI), projetada em Python 3 para centralizar e organizar demandas de forma ágil, padronizada e livre de distrações. O sistema resolve a desorganização ao exigir um fluxo estruturado de entrada: garante que toda tarefa possua um título válido e uma classificação estrita de prioridade (Alta, Média ou Baixa), atribui automaticamente o status inicial como *Pendente* e disponibiliza uma consulta imediata e legível de todas as atividades cadastradas no terminal.

---

2. Decomposição do Problema e Dados do Sistema

Para garantir integridade no armazenamento em memória (via lista de dicionários em Python) e previsibilidade na execução, o sistema é decomposto em entradas estruturadas, saídas padronizadas e regras de negócio inegociáveis:

Categoria	Elemento / Campo	Detalhamento Técnico e Exemplos
Entrada de Dados	**Título da Tarefa** (String)	Identificador curto e obrigatório da atividade (ex.: *"Implementar validação no main.py"*).
Entrada de Dados	**Descrição** (String)	Detalhamento livre sobre os passos ou contexto da tarefa (ex.: *"Criar laço while para impedir títulos em branco"*).
Entrada de Dados	**Prioridade** (String)	Grau de urgência da tarefa, restrito exclusivamente às opções **Alta**, **Média** (ou *Media*) e **Baixa**.
Entrada de Dados	**Data Limite** (String)	Prazo planejado para conclusão da atividade no formato DD/MM/AAAA ou texto livre (ex.: *"25/10/2026"*).

Categoria	Elemento / Campo	Detalhamento Técnico e Exemplos
Atributo Automático	**Status Inicial** (String)	Atribuído automaticamente pelo sistema como **"Pendente"** no instante do cadastro.
Saída de Dados	**Menu e Mensagens de Feedback**	Exibição contínua das opções (1-Cadastrar, 2-Visualizar, 3-Sair), alertas claros de erro de digitação e confirmação de sucesso ao salvar.
Saída de Dados	**Relatório de Tarefas no Terminal**	Listagem numerada exibindo Título, Descrição, Prioridade, Data Limite, Status e totalizador (ou aviso claro quando a lista estiver vazia).

Regras de Negócio do Sistema

1. **Proibição de Título Vazio:** O sistema não aceita tarefas sem título ou compostas apenas por espaços em branco. Caso o usuário pressione *Enter* sem digitar um texto válido, o programa exibe uma mensagem de erro e solicita o título novamente.
2. **Validação Estrita de Prioridade:** O campo de prioridade aceita exclusivamente os valores **"Alta"**, **"Média"** (ou **"Media"** sem acento) e **"Baixa"**, normalizando letras maiúsculas e minúsculas. Qualquer valor diferente gera alerta imediato e repete a pergunta.
3. **Padronização Automática de Status:** O usuário não precisa digitar o status ao criar uma tarefa; o sistema define automaticamente o valor **"Pendente"** no dicionário da tarefa recém-criada.
4. **Tratamento Seguro de Lista Vazia:** Ao acionar a opção de visualização quando ainda não há tarefas registradas, o sistema não gera falha nem exibe tela em branco; ele informa explicitamente que nenhuma tarefa foi cadastrada até o momento.

---

3. Receita Lógica do Sistema (Algoritmo em Português)

O algoritmo abaixo descreve de forma sequencial e clara todo o funcionamento lógico que guia a implementação em Python no arquivo `src/main.py`:

A) Fluxo Principal (Loop do Menu de Navegação)

1. Iniciar o programa criando uma lista vazia chamada `tarefas` para guardar os registros em memória.
2. Iniciar um laço de repetição contínuo que exibe na tela o cabeçalho do **TaskTracker** e as três opções: **[1] Cadastrar nova tarefa**, **[2] Visualizar tarefas cadastradas** e **[3] Sair da aplicação**.
3. Ler a opção escolhida pelo usuário:
   - Se a opção for **1**: executar a receita de *Cadastro de Tarefa*.
   - Se a opção for **2**: executar a receita de *Visualização de Tarefas*.
   - Se a opção for **3**: exibir mensagem de encerramento e finalizar o programa.
   - Se for qualquer outro valor: exibir aviso de *"Opção inválida! Escolha 1, 2 ou 3"* e mostrar o menu novamente.

B) Passo a Passo do Cadastro de Tarefa

1. Solicitar ao usuário que digite o **Título da tarefa** e remover espaços extras nas pontas.
2. Verificar se o título está vazio: enquanto estiver em branco, exibir *"Erro: O título da tarefa é obrigatório e não pode ficar vazio"* e pedir o título novamente.
3. Solicitar ao usuário que digite a **Descrição da tarefa** (caso deixe em branco, registrar como *"Sem descrição detalhada"*).
4. Solicitar ao usuário que informe a **Prioridade** (Alta, Média ou Baixa).
5. Verificar se a prioridade informada corresponde a *Alta*, *Média*, *Media* ou *Baixa*: enquanto o valor digitado for diferente dessas três opções, exibir *"Erro: Prioridade inválida! Digite apenas Alta, Média ou Baixa"* e repetir a pergunta.

6. Solicitar ao usuário a **Data Limite** para conclusão (ex.: DD/MM/AAAA; caso não informada, registrar *"Não definida"*).
7. Definir automaticamente o campo **Status** da tarefa como `"Pendente"`.
8. Montar um dicionário reunindo `titulo`, `descricao`, `prioridade`, `data_limite` e `status`, e adicioná-lo à lista `tarefas`.
9. Exibir mensagem de confirmação de sucesso na tela e retornar ao menu principal.

C) Passo a Passo da Visualização de Tarefas

1. Acessar a lista `tarefas` armazenada em memória.
2. Verificar se a lista possui algum item cadastrado:
   - **Se a lista estiver vazia:** exibir a mensagem *"Nenhuma tarefa cadastrada no momento. Utilize a opção 1 para adicionar sua primeira tarefa."*
   - **Se a lista possuir tarefas:** percorrer a lista do primeiro ao último item, exibindo de maneira organizada o número da tarefa, Título, Descrição, Prioridade, Data Limite e Status, além do total de tarefas exibidas no final.
3. Retornar ao menu principal para nova operação.

---

4. Mapeamento Arquitetural para Git e GitHub (Fase 2)

A organização física do projeto segue boas práticas de Engenharia de Software, separando documentação, regras de versionamento e código-fonte na seguinte árvore de diretórios:

```
tasktracker-projeto/
├── README.md                  # Apresentação do projeto e instruções de uso
├── .gitignore                 # Ignora arquivos temporários (__pycache__/, .venv/)
├── docs/
│   └── planejamento_logico.pdf # Planejamento lógico entregue na Fase 1
└── src/
    └── main.py                # Código-fonte principal da aplicação em Python 3
```

Plano de Aprendizado e Versionamento com Git e GitHub

Na implementação prática com Git e GitHub, o objetivo é aplicar o controle de versão de maneira incremental e profissional, registrando cada etapa evolutiva do software por meio de commits atômicos e semânticos. A jornada contempla a inicialização do repositório local (`git init`), a proteção do repositório contra arquivos temporários do Python via `.gitignore`, o registro separado entre a estrutura documental inicial, a construção do menu principal em Python e a implementação das regras de validação de negócio, culminando na publicação remota (`git push -u origin main`) para garantir rastreabilidade completa do código e preparar a base para a futura containerização com Docker na Fase 3.